Title: Log-structured Storage Engines: Toward Automated Vulnerability Detection in High-performance Computing Clusters

Year: 2015

Authors: N. Hoffmann, B. Singh (National Laboratory for Applied Computation)

Keywords: cloud computing, microservices, containers, performance engineering, distributed systems

Abstract:

Automated vulnerability detection remains a critical challenge for database indexing, particularly in high-performance computing clusters. This paper proposes a framework built on log-structured storage engines that explicitly accounts for configuration drift detection. We evaluate the framework on the TPC-H benchmark and compare it against 7 baseline methods. Experimental results show that the proposed approach improves 99th-percentile latency by 23.5% while keeping defect detection rate within acceptable bounds. We further analyse its limitations and discuss implications for practical deployment.

1. Introduction

The rapid maturation of database indexing has changed how researchers and practitioners approach automated vulnerability detection, yet practical deployment is still limited by scalability, cost, and reproducibility concerns. Over the past decade, log-structured storage engines have attracted sustained attention as a promising route toward solving automated vulnerability detection in realistic settings such as high-performance computing clusters. Despite considerable progress, existing solutions for automated vulnerability detection tend to degrade when moving from controlled laboratory conditions to production workloads, which motivates a more systematic investigation. This paper addresses these gaps by studying how log-structured storage engines interact with serverless execution runtimes when applied to automated vulnerability detection.

Our work is motivated by the growing demand for high-performance computing clusters, where reliability and transparency requirements are especially strict. We make three contributions: a unified formulation of automated vulnerability detection, a practical design built on log-structured storage engines, and an empirical evaluation on the TPC-H benchmark. Understanding configuration drift detection is equally important, because it shapes both the theoretical limits and the operational cost of any proposed approach. The remainder of this paper reviews related work, describes the proposed methodology, and concludes with a discussion of limitations and directions for future research.

2. Related Work

Ahmed et al. (2011) introduced an early framework based on log-structured storage engines, reporting a 30.8% improvement in resource utilization over previous baselines. Subsequent work by Vasquez et al. (2014) extended this line of research to high-performance computing clusters, although the evaluation was restricted to small-scale settings. A survey by Haddad et al. (2009) categorised existing approaches to automated vulnerability detection into three families and highlighted the lack of standard benchmarks. Tanaka et al. (2008) analysed configuration drift detection and showed that purely heuristic solutions cannot guarantee consistent request throughput. More recently, OConnor et al. (2014) combined log-structured storage engines with serverless execution runtimes, achieving a 7.8% reduction in overhead on the TPC-H benchmark.

Several studies have criticised the reliance on synthetic workloads, arguing that results on the TPC-H benchmark may not generalise to high-performance computing clusters. In contrast to these efforts, our work treats automated vulnerability detection and configuration drift detection jointly rather than as independent concerns. Industry reports corroborate these academic findings, pointing to automated vulnerability detection as a persistent obstacle for high-performance computing clusters. Despite this body of literature, no prior study has

systematically compared log-structured storage engines and serverless execution runtimes under identical experimental conditions.

3. Methodology

We formalise automated vulnerability detection as a constrained optimisation task in which the objective is to maximise 99th-percentile latency while keeping defect detection rate within a prescribed budget. The proposed architecture consists of four stages: data acquisition, preprocessing, core processing based on log-structured storage engines, and a validation layer that incorporates serverless execution runtimes. All experiments were conducted on the TPC-H benchmark, which provides 8 labelled instances or configurations covering a broad range of operating conditions. To ensure reproducibility, every experiment was repeated 5 times with independent random seeds, and we report mean values together with 95% confidence intervals.

Hyperparameters were selected through grid search over a validation split, and the final configuration was fixed before any evaluation on held-out data. The core routine applies log-structured storage engines to reduce the effect of configuration drift detection, followed by a calibration step that stabilises 99th-percentile latency. Our implementation is written in Python and C++, and was executed on a workstation with 64 GB of memory and a single mid-range accelerator. We compare against 7 baseline methods, including a conventional approach built on serverless execution runtimes and two recent variants reported in the literature.

The proposed design improves 99th-percentile latency by 23.5% relative to the strongest baseline and reduces defect detection rate by 7.6%, a statistically significant difference ($p < 0.05$). Ablation experiments show that removing the serverless execution runtimes component reduces 99th-percentile latency by approximately 5.8%, confirming its contribution. Threats to validity include the limited diversity of the TPC-H benchmark and the possibility that deployment in high-performance computing clusters introduces conditions not captured in our setup.

4. Conclusion

This paper investigated automated vulnerability detection through the lens of log-structured storage engines, and demonstrated that a carefully designed pipeline can improve 99th-percentile latency by 23.5% on the TPC-H benchmark. The findings suggest that database indexing will continue to benefit from closer integration of log-structured storage engines and serverless execution runtimes, particularly in high-performance computing clusters. Several limitations remain, including the restricted scale of our evaluation and the simplified treatment of configuration drift detection. Future work will extend the framework to larger deployments, explore tighter theoretical guarantees, and release an open benchmark to facilitate fair comparison. We hope the evidence presented here encourages the community to adopt more rigorous and transparent evaluation practices for automated vulnerability detection.